// INTERVIEW KA SAWAAL · GUIDE

Why 0.1 + 0.2 ≠ 0.3 (Floating Point)

@thestackexplained
FREE PDF GUIDE



TL;DR Doubles keep only **53 bits**, so 0.1 and 0.2 get rounded: `0.1+0.2` = `0.30000000000000004`. Never `==` floats; money = **integer cents**.

01 Concept

A floating point number is binary scientific notation: **sign × fraction × 2^exponent**. Almost every language uses **IEEE 754 binary64** (a double): 1 sign bit, 11 exponent bits, 52 fraction bits (+1 implicit leading bit = 53 bits, ~15–17 decimal digits).

In base 2 a fraction is finite only if its denominator is a power of 2. 0.5, 0.25, 0.375 are exact; **0.1 = 1/10** is not (10 = 2×5), so it becomes `0.000110011…` repeating forever and must be rounded. Analogy: 1/3 in decimal is 0.333... and you have to stop somewhere.

IEEE 754 double = 64 bits



sign 1 exponent 11 fraction 52 (+1 implicit)

`0.1 → 0.1000000000000000055511… (rounded)`

Fixed 53-bit precision: any value needing more bits is rounded to the nearest double.

02 How it works

1. Literal `0.1` is rounded to the nearest double — slightly **above** 0.1 (≈5.55e-18).
2. Literal `0.2` is rounded too — also slightly above 0.2 (≈1.11e-17).
3. The sum is computed exactly and rounded once more → `0.3000000000000000444…`, printed as `0.30000000000000004`.
4. Literal `0.3` rounds slightly **below** 0.3. Different doubles, so `==` is false. No bug — each step is correctly rounded.

Need	Use	Why
Physics, ML, graphics	float / double	Fast, tiny relative error is fine
Compare computed floats	abs(a-b) ≤ tol	`==` fails after rounding
Money, billing	int cents / Decimal	Exact base-10 arithmetic
Huge sums	Kahan / math.fsum	Limits error accumulation

03 Real-world implementation

✓ DO THIS IN PRODUCTION

- Store prices as **integer minor units** (paise/cents) in DB and APIs; format only at display time.
- Use `DECIMAL(p,s)` columns in SQL, `decimal.Decimal` in Python, `BigDecimal` in Java.
- In tests, assert with tolerance: `math.isclose`, `pytest.approx`, `assertEquals(a, b, 1e-9)`.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> import math; math.isclose(0.1 + 0.2, 0.3)
True
>>> from decimal import Decimal
>>> Decimal('0.1') + Decimal('0.2') == Decimal('0.3')
True
>>> 10 + 20 == 30   # cents
True
```

⚠ COMMON PITFALLS

- `Decimal(0.1)` (from a float) keeps the error — build from a string `Decimal('0.1')`.
- A fixed absolute tolerance breaks for very large/small values — use a relative tolerance.
- JavaScript `Number` is always a double; use integers or a decimal library for money.

04 Interview Q&A

Q. Is 0.1 + 0.2 == 0.3 true?
No. Both literals and the sum are rounded to 53-bit doubles; the sum lands on a different double than 0.3.

Q. Why is 0.5 + 0.25 == 0.75 true?
All three are exact in binary (denominators are powers of 2), so no rounding happens.

Q. How many digits of precision does a double have?
53 significant bits ≈ 15–17 decimal digits.

Q. How do you compare floats safely?
Relative + absolute tolerance, e.g. `math.isclose(a, b, rel_tol=1e-9)`.

SCENARIO **Q. Your payments service shows ₹1999.9999999998 totals. Fix?**
Move to integer paise (or DECIMAL) end-to-end, migrate stored floats with rounding, and format only at display.



// QUICK REVISION

- Double = 1 + 11 + 52 bits, 53 significant bits.
- 0.1 has no finite binary form → rounded.
- Never [redacted] on computed floats.
- Money = integer cents or Decimal.